Supplementary material for

"The Model Knows the Textbook but Not Your Class: Curriculum Knowledge Graph Constraints for Slide Generation in Two Vocational-Education Courses"

Suxing Liu

This document holds eleven evidence and reference blocks that the main text summarises: the ablation results (Section S1), the mutation test of the graph's own structural checks (Section S2), the behavioural probe of the generated code (Section S3), the replication of the claim-bearing conditions on a second vendor's model (Section S4), the concept-level paired analysis behind the per-baseline tests in §5.4 of the main text (Section S5), the full qualitative readings behind the two cases in §5.8 of the main text (Section S6), the verbatim prompt of one session (Section S7), one raw model response with the validator's view of it (Section S8), the seven measurement defects with symptom, consequence and fix (Section S9), the recomputation of the two coverage conventions (Section S10), and the reproduction commands (Section **??**). All numbers are regenerated by the scripts named in each section from the archived raw outputs; nothing here is hand-transcribed.

S1 Ablation of the contract

Each ablation arm retains the complete contract and rule wording of `full` and deletes exactly one data block, together with the rule text that would leak the deleted information back in: `no_order` removes the ordering block and rule 3, `no_misconception` removes the misconception block and rule 5, and `no_forbidden` removes the forbidden-term block and rule 6. The rule text must be removed together with the data—deleting only the data while keeping a rule that says "you must follow the given order" forces the model to guess an order, and the ablation is no longer clean; a unit test guards this. All four rows come from the ablation batch in `outputs/bench_ablation/`—three sessions × three resamplings, $n = 9$ per arm—whose own nine `full` re-runs anchor the table. This batch is *separate* from the main experiment, and comparisons are made within the table; on coverage and on misconception coverage its `full` arm nonetheless lands on the same values as the main batch (0.981 and 1.000), which is the agreement the main text relies on when it places this arm beside the 2×2.

One arm in this table is a *null manipulation* and is kept only for completeness. Deleting the ordering block leaves the required-concept list, which is itself emitted in topological order—the same sequence the deleted block restates—so `no_order` still receives the order. Its 0.000 violation rate is therefore not evidence that an ordering constraint is unnecessary; it says only that a second statement of the order can be removed without effect. A clean test of the constraint would have to shuffle the concept list, which we did not do.

Regenerated by `python paper/tables.py` from `outputs/bench_ablation/`.

Table S1: Ablations; each removes exactly one part of the graph. Mean (standard deviation) over nine runs, all from the ablation batch.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑
`full`	0.981 (0.056)	0.000 (0.000)	0.026 (0.063)	1.000 (0.000)
− ordering	**1.000** (0.000)	0.000 (0.000)	0.000 (0.000)	1.000 (0.000)
− misconceptions	0.944 (0.083)	0.000 (0.000)	0.043 (0.065)	**0.111** (0.333)
− forbidden terms	**1.000** (0.000)	**0.083** (0.125)	0.000 (0.000)	1.000 (0.000)

S2 Mutation test of the graph's structural checks

The shipped graph returns zero findings on all ten checks, which on its own is weak evidence: a checker that never fires cannot be distinguished from one that cannot fire. Each of the twelve codes the validator can emit is therefore exercised by one minimal defect injected into a copy of the graph, and Table S2 records whether the owning code responds and what responds with it. All twelve codes fire on their own mutant. A single mutant triggers a second code—an ungrounded skill also violates check 3—which is the correct verdict rather than interference, so the checks are specific as well as sensitive. Regenerated by `python paper/graph_checks.py`.

Table S2: Each of the twelve codes is exercised by one minimal defect injected into a copy of the shipped graph. "Also fires" lists codes that respond in addition to the intended one; a check that failed to fire here would be decorative.

Chk	Code	Injected defect	Also fires
1	`DANGLING_EDGE`	add relates_to edge to undefined node C9999	–
2	`PREREQ_CYCLE`	reverse prerequisites_for C1001→C1003	–
3	`NOT_GROUNDED`	add Concept with no appears_in edge	–
4	`MISSING_EVIDENCE`	strip evidence from is_a C1101→C1001	–
5	`UNIT_CAPACITY`	add 6 filler concepts to U12 (5 present, cap 8)	–
5	`NO_CORE_CONCEPT`	downgrade the 3 mastery concepts of U12	–
6	`DUPLICATE_NAME`	rename C1204 to the name already used by C1001	–
7	`FORBIDDEN_REQUIRED_OVERLAP`	forbid 'circular queue' in U11, where it is a core concept	–
8	`FORBIDDEN_SIBLING_OVERLAP`	forbid 'false overflow' in U11, where it is a sibling recap term	–
9	`ORPHAN_MISCONCEPTION`	add Misconception with no addresses_misconception in-edge	–
10	`IMPLEMENTS_TARGET_TYPE`	point implements at a Misconception node	–
10	`IMPLEMENTS_SOURCE_UNGROUNDED`	implements from a Skill with no appears_in edge	`NOT_GROUNDED`

Five unit tests in `tests/test_graph.py` pin the four self-consistency checks (7–10) to mutants of the shipped graph, so a regression in any of them fails the suite.

S3 Behavioural probe of the generated code

The probe (`paper/behavior_check.py`) extracts every code block from the 54 main runs, strips top-level demo statements with an AST filter—otherwise a demonstration loop leaves `front=2, rear=5` in the namespace and every later probe fails for the wrong reason—executes the remainder in a fresh namespace, and interrogates up to four behaviours: first-in-first-out order, the empty-

dequeue signal, the full signal, and survival of repeated enqueue. A block qualifies only if it defines a module, a class, or a function whose first parameter is a queue; fragments such as a bare `enqueue(self, x)` method body are not testable. Class probes are scored separately for bounded and unbounded implementations: an unbounded queue passes the full probe if 64 enqueues complete without error, a bounded one if it eventually raises after at least one successful enqueue. A probe counts as passed only when the code returns a boundary signal (a sentinel, `False`, a raised exception, or a printed message); a silent wrong answer fails.

Table S3: Behavioural probe over all 117 code blocks from the 54 main runs. "Testable" is the share of blocks the harness can execute at all; the rate is the share of testable blocks that satisfy every behaviour asked of them.

Condition	Blocks	Testable	Passed	Rate
`full`	28	9	8	0.89
`rag_mc`	18	6	6	1.00
`rag`	19	6	6	1.00
`placebo`	18	3	3	1.00
`cot`	18	7	7	1.00
`zero_shot`	16	6	6	1.00
All	117	37	36	0.97

The single failure is a genuine scaffolding defect rather than a probe artefact: in the U10 `full` run 3 deck, a program-style block writes `enqueue(q, x)` against a capacity constant `CAP` that its own slide never defines. This is precisely the "slides are deliberately partial" argument of the main text: the excerpt assumes a surrounding program that a slide does not carry.

As the main text argues, this is a bound and not a metric. The harness is authored by the evaluators, so the evaluator supplies part of the answer it then checks; and the qualifying denominator (0.32 of blocks, unevenly spread) is too small and too condition-dependent to rank arms. `full` passing 8 of 9 against 28 of 28 for the baselines reads as a regression but is not evidence of one.

S4 Cross-model replication

The same generator, prompts, sessions and validator, run on a second vendor's model. This block exists to answer one question—does the contrast that carries the paper (`full` against `rag_mc`) keep its sign off the model it was developed on—and it is not a comparison of model quality.

Two settings differ and are recorded per run in `outputs/bench_kimi/results.csv`: the backbone (`kimi-k3`) and the sampling temperature. Kimi K3 rejects any temperature other than 1.0, so the replication runs at 1.0 against 0.3 for the main table; the value is pinned in `ProviderPreset.fixed_temperature` rather than left to the caller's default, and a unit test holds it there. Within-model contrasts are therefore unaffected, and we compare deltas across models, never levels.

Runs: three sessions × two resamplings, six per condition, except `zero_shot` on Kimi ($n = 5$): one cell could not be generated because the vendor account reached its credit limit, and it is reported as missing rather than refilled. `rag` and `cot` were not re-run on the second backbone; they carry none of the claim and each extra condition costs about seven minutes at the account's concurrency limit of one.

Regenerated by `python paper/tables.py -cross outputs/bench_kimi`.

Table S4: The same conditions under two backbones. Mean (standard deviation) over runs; `rag` and `cot` were not re-run on the second backbone. DeepSeek V4 at temperature 0.3, Kimi K3 at 1.0 (vendor-pinned), so levels are not comparable across the two blocks—only deltas within a block are.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑	Pass
DeepSeek V4					
`full`	0.981 (0.056)	0.000 (0.000)	0.020 (0.045)	1.000 (0.000)	67%
`rag_mc`	0.698 (0.295)	0.111 (0.220)	0.007 (0.022)	0.556 (0.527)	22%
`rag`	0.659 (0.248)	0.111 (0.220)	0.030 (0.089)	0.222 (0.441)	0%
`placebo`	0.675 (0.266)	0.167 (0.250)	0.096 (0.131)	0.333 (0.500)	0%
`cot`	0.614 (0.185)	0.019 (0.056)	0.089 (0.183)	0.389 (0.486)	0%
`zero_shot`	0.643 (0.248)	0.019 (0.056)	0.037 (0.089)	0.222 (0.441)	0%
Kimi K3					
`full`	1.000 (0.000)	0.000 (0.000)	0.016 (0.025)	1.000 (0.000)	67%
`rag_mc`	0.734 (0.229)	0.056 (0.136)	0.033 (0.082)	0.667 (0.516)	17%
`placebo`	0.659 (0.188)	0.056 (0.136)	0.028 (0.068)	0.333 (0.516)	0%
`zero_shot`	0.543 (0.270)	0.000 (0.000)	0.000 (0.000)	0.400 (0.548)	0%

What replicates. On both metrics that carry the argument, and in the same direction: misconception coverage 1.000 → 0.667 on Kimi against 1.000 → 0.556 on DeepSeek, coverage 1.000 → 0.734 against 1.000 → 0.698. The variance behaviour replicates more exactly than the means do—`full` is 1.000 with zero standard deviation in every run under both models, and `rag_mc`'s standard deviation is 0.516 against 0.527.

What does not. The gain shrinks on the second model (−0.333 against −0.444 for misconceptions, −0.266 against −0.283 for coverage), and on U11 every condition scores 1.000 on Kimi, zero-shot included, so one of the three sessions contributes no contrast at all. The pass-rate gap (67% against 17%) rests on six runs per arm and carries no usable interval. Two backbones are two draws from a large space: this rules out an artefact of one vendor's tuning and supports no statement about model families.

S5 Concept-level paired analysis

The main text reports coverage per deck, which leaves three clusters and a clustered interval that includes zero. The contract, however, is written per concept, and the same decks can be scored that way: every required concept is either named in a deck or not, so each (session, resampling, concept) triple is a paired binary observation shared by all conditions. Sixteen required concepts across three sessions and three resamplings give $N = 48$ pairs per baseline.

The matching is the shipped one (validator surface forms and graph aliases, body pages only), so this section adds no new metric—only a different unit of analysis. The test is the exact McNemar test on the discordant cells; the per-unit counts are printed in the main text because the pairs are *not* independent (sixteen concepts nested in three sessions, and the three resamplings of a cell repeat the same concept list), so the p values describe the consistency of the direction, not a population parameter.

Reading the pairs. Two features matter. First, every baseline puts more discordant cells on the side of `full` than on its own—11 to 1 against `rag_mc`, 17 to 1 against `cot`—and no baseline reverses the sign, so the direction of the coverage effect does not depend on which control is chosen. Second, no single concept carries it: dropping any one concept from the metric moves a session's gap by at most 0.056, including U12, where three concepts alone produce a gap between

Table S5: Concept-level pairs against each baseline, $N = 48$ per row. "both" = covered by both conditions, "`full` only" / "base only" = the discordant cells, RD = paired risk difference. Regenerated by `python paper/concept_pairs.py`.

Baseline	N	both	`full` only	base only	RD	p
`rag_mc`	48	36	11	1	+0.208	6.3×10^{-3}
`rag`	48	34	13	1	+0.250	1.8×10^{-3}
`placebo`	48	35	12	1	+0.229	3.4×10^{-3}
`cot`	48	30	17	1	+0.333	1.4×10^{-4}
`zero_shot`	48	33	14	1	+0.271	9.8×10^{-4}

Table S6: Leave-one-concept-out: the deck-level coverage gap against `rag_mc` is recomputed with each required concept removed in turn, per session. The range is the answer to "is one easy concept carrying the result".

Session	gap, all concepts	range over drops	concepts
U10	−0.056	$[-0.067, 0.000]$	6
U11	+0.238	$[+0.111, +0.278]$	7
U12	+0.667	$[+0.500, +1.000]$	3

+0.500 and +1.000. What the pairs do *not* fix is U10: its discordant cells are 0 to 1 against `rag_mc` and 3 to 1 against `zero_shot`, so the session-level sign can still flip there, exactly as the clustered interval in the main text says.

S6 Qualitative readings of two cases

A positive case: `full` on U12 U12 is the hardest session: it requires choosing between a circular and a linked queue, a judgement task with no single correct answer, and one required concept ("application scenarios of queues") is abstract and easily reduced to platitudes. All three `full` runs reached coverage 1.000 and each produced a genuine comparison slide (predictable capacity and frequent queries favour the circular queue; variable length and frequent enqueue–dequeue favour the linked queue), handling the breadth-first-search misconception on a dedicated counter-example slide. Coverage for the five baselines on the same session is **0.333**: they cover only "linked queue" and dispose of selection and scenarios in one clause.

A negative case: `full` fails too In the third resampling of U10, coverage is 0.833, missing the concept "queue" itself. Reading slide by slide: the title slide states that a queue is a linear list inserting at one end and deleting at the other, but the first body slide is titled "head, tail and the sequential queue," and the deck never defines a queue separately again. The model assumed students would remember the definition from one sentence on the cover, and nothing in the following five slides reinforces it.

What kind of failure this is matters. Whether a definition on the title slide counts as teaching is a pedagogical convention, and the omission disappears if the title slide is admitted (coverage rises to 1.000); anyone who thinks a cover definition suffices is entitled to read this run as a pass. What the metric records is narrower: the definition is stated once, in a position the validator does not treat as instruction, and never returned to. The validator flagged it, which is the point of closing the loop—the method does not guarantee correctness, it guarantees that **errors are not silently passed over**, and it leaves the disposition to the teacher.

A second failure is convention-independent and therefore more instructive: the second resampling of U10 contains two prerequisite inversions, the model teaching the sequential queue

first and returning to head and tail afterwards. On the metrics this is a 0.133 violation rate; in the classroom it means students first encounter the head and tail indices without knowing what they denote. The topological order was in the prompt and the model violated it anyway, which is why we insist on post-hoc validation rather than assuming that saying it implies doing it.

S7 The prompt for one session

The course is taught in Chinese, so the prompt is written in Chinese; that is a property of the deployment, not a choice we made for the experiment. Verbatim prompts for the `full` and `rag_mc` conditions of session U11 are archived as `paper/prompt-U11-full.txt`; the rendering below keeps every section header, every constraint and every hard rule, and condenses only the per-item lists that the archive reproduces in full.

> **Instruction.** Generate teaching slides according to the following constraints.
>
> **Session.** U11 Circular queue (7 pages).
>
> **Teaching objective.** Understand how a circular queue reuses space through modular arithmetic, distinguish the empty and full conditions correctly, and write the implementation independently.
>
> **Concepts that must be covered (none may be omitted).** Seven entries, each with identifier, name, importance level, optional complexity and a one-sentence definition, for example: `C1104 Full condition (proficient)`: to separate "full" from "empty" one storage cell is usually sacrificed, so the queue is full when `(rear + 1) % maxsize == front`.
>
> **Explanation order (mandatory; this is the topological order induced by the prerequisite relation).** The seven identifiers in sequence, with arrows.
>
> **Misconceptions that must be corrected.** One entry: the erroneous claim (the queue is full when `rear == front`), followed by the correction and the consequence of confusing the two.
>
> **Accompanying lab.** Implement a circular queue of capacity 8, run 20 enqueues and 15 dequeues, and verify modular reuse and the full condition.
>
> **Evidence anchors (for teacher verification, not generated content).** Six entries of the form "`C1101: handout §4.3: the circular queue exists precisely to solve false overflow`".
>
> **Previously taught concepts that may be recapped in one sentence but not expanded.** Ten sibling concept names, followed by the constraint that they must not exceed 25% of the text.
>
> **Concepts that must not be introduced.** Five names (priority queue, monotone queue, message queue, shortest path, topological sort).
>
> **Output format.** A single JSON object with `unit_id` and a `slides` array; each slide carries `index`, `layout` (one of seven), `title`, `bullets`, `code`, `notes`, `concept_ids` and `background`. No Markdown fence, no commentary.
>
> **Hard requirements.** (1) Seven pages, first a title page and last a summary. (2) Every concept in the mandatory list must appear in some page's `concept_ids`. (3) Page order must follow the given explanation order; no prerequisite may be inverted. (4) At most four bullets per page, at most forty characters each. (5) If misconceptions are listed, at least one page must have layout `misconception`. (6) Do not introduce forbidden concepts; if one must be mentioned, set `background` to true.

The `rag_mc` control replaces everything from *Session* to *Forbidden concepts* with a flat list of the same definitions under the heading *Textbook excerpt*, and keeps the output format and hard rules identical. That is the point of the control: the information is held constant and only its structure changes.

S8 One raw response, and what the validator saw

Responses are Chinese for the same reason the prompts are. The first two slides of one `full` response (U11, run 1) are: index 1, layout `title`, "Circular queue", three bullets naming the lecture, the objective and the lab, no code, `concept_ids [C1101]`; index 2, layout `concept`, "The circular queue and modular arithmetic", four bullets on vacated cells, the logical ring, `(index + 1) % maxsize` and the $O(1)$ bound, code carrying the two pointer-advance lines. The validator records coverage 1.000, prerequisite violation 0.000, misconception coverage 1.000, out-of-graph 0.000, background 0.012 against a 0.25 ceiling, and editability 1.000, with no findings. The report is addressed to concepts, not to a score: a failure names the uncovered concept ids, the out-of-graph terms and the inverted pairs, so a low number can be traced to a slide rather than only observed. The verbatim JSON and report are in the archive under `outputs/main_v1/`.

S9 The seven measurement defects

The main text states the count, the common structure and the two readings a reader should carry away; this section is the record. Seven defects were found during the study and fixed before any number reported in the paper was produced. None of them raised an error, and none was found by reflection: each came from printing model outputs slide by slide and reading them against the metric that scored them.

Table S7: The seven measurement defects that silently favour the constraint-based condition, and their fixes. Column "Symptom" names the surface observation; "Consequence" the wrong number it produced.

#	Defect	Symptom	Consequence	Fix
1	Verbatim coverage matching	Graph says "enqueue and dequeue operations", baseline writes "enqueue and dequeue"	Baseline coverage crushed to 0.50 while `full` scores 1.00 by copying source wording	Add aliases to the graph; one shared vocabulary across conditions
2	Order scored from self-reported IDs	Baselines never fill `concept_ids`	Empty sequence gives 0.00 violation; **an unanswered script scores full marks**	Score from textual position instead
3	Preview slides counted as body	Title and objectives slides list every concept	Ordering determined by list order rather than teaching order; three spurious violations in U10	Exclude title, objectives and summary
4	Forward reference treated as teaching	One slide remarks that modular arithmetic is key to implementing the circular queue	The concept is located three slides early; two spurious violations in U11	Locate by title naming first, fall back to first mention
5	Cross-slide subtraction	The simple term is cancelled by a compound term on another slide	The concept is scored as never taught although the deck does teach it	Compute net occurrences per line
6	Misconception anchors too broad	Unit-level `addresses_misconception` used as a slide anchor	Any slide with the same layout counts as corrected; the ablation still scores 1.00	Restrict anchors to concept level; fix one mis-attached data edge
7	Out-of-graph test too strict	Baselines invent their own ID scheme	Out-of-graph rate inflated to 0.67 when the only difference is a namespace	Count only non-existent IDs matching the graph's ID format

Three notes the table cannot carry. First, **the common structure across all seven is that the representation the constrained method receives is exactly the representation the metric uses**—this is the fairness problem of evaluating a structured condition with unstructured metrics, and it is structural rather than accidental, which is why it took seven separate sightings to see.

Second, every row is now pinned by a regression test in the file `test_measurement_defects.py` under `tests/`: each test builds the deck that used to fool the metric and asserts the value it should produce, and where the wrong arithmetic can be written down, asserts that arithmetic too. The full suite stands at 44 tests.

Third, defect 6 is the one that generalises beyond this study. It was not a bug in metric code but **an error in the graph data**: one mis-attached "related" edge made any misconception-layout slide mentioning empty-queue detection count as addressing a different misconception, so the ablation still scored full marks and the conclusion about misconception injection nearly became false. **Graph errors silently contaminate metrics**—a risk specific to graph-driven methods, where the knowledge base is simultaneously the object of study and part of the measuring instrument. We therefore recommend reporting how the metrics behave *on the baselines* and doing at least one manual pass with outputs printed slide by slide.

S10 Sensitivity of the coverage convention

The coverage convention has two halves—body slides only, and matching on name and aliases—and each is a decision rather than a fact about teaching, so each was recomputed over all 54 main runs. Regenerated by `python paper/coverage_sensitivity.py`; the per-run values are in `paper/coverage_sensitivity.csv`.

Admitting the preview pages. Keep the title slide, and coverage moves in only two runs; the ranking of the six conditions is unchanged (Spearman $\rho = 1.00$; $\rho = 0.94$ once objectives and summary are also admitted), and the headline contrast against `rag_mc` widens rather than shrinking ($+0.283 \rightarrow +0.302$). The convention is therefore conservative with respect to the claim: admitting the pages it excludes would strengthen the reported advantage, not weaken it.

Name-only matching. The aliases decision points the same way, and more sharply. Recomputing coverage with name-only matching drops baseline coverage to 0.325–0.389 while `full` falls only to 0.963, because `full` was handed the canonical strings and copied them (thirty-one of the 54 runs move; the largest single drop, from 1.000 to 0.143). Alias matching therefore *narrows* the reported gap: the conservative number for `full` against `rag_mc` is $+0.283$, and the gap a stricter matcher would produce is $+0.574$. This is the opposite of a defect that manufactures the result—it is a correction that runs against the hypothesis, which is why we keep it and why the reported advantage should be read as a lower bound rather than a best case.

S11 The repair loop

The validator emits findings, and in the deployed workflow a teacher acts on them; in the main experiment nothing did. One third of the `full` runs contain at least one error-level finding, so we ran a repair loop on the stored runs of two cells: the DeepSeek grid of the main results table (nine runs, three failures) and the `agnes-2.5-flash` cell of the sweep (row 3; six runs, two failures)—the cell where the misconception slot's advantage disappears, and therefore the harder place to ask.

The policy is fixed and deliberately plain: for a run with an error-level finding, the validator's error findings are appended to the same condition prompt and the whole deck is regenerated, up to three attempts; first-pass runs are untouched, so the repaired count shares its denominator with the first-pass count. The appended text names the uncovered concepts and the inverted pairs and adds nothing else—for `full` those names are already in the contract, so a finding restates a requirement instead of supplying one. The same loop on `zero_shot` would hand over the syllabus that condition exists to lack, which is why it is not run there: whether a repair loop is fair is a property of the condition, not of the loop.

On the main grid the loop closes the gap. All three failures were repaired in a single round each: 6/9 → 9/9, mean coverage 0.981 → 1.000, three model calls in total. The repairs are substantive: a missing concept was given the page it lacked (in U10/r2 the prerequisite `C1002` existed only inside a later summary bullet), and an inverted pair was fixed by splitting a page that had merged two concepts (U11/r3), so the metrics are computed on more content than before, not less.

On the third backbone it does not. The same policy on the `agnes-2.5-flash` cell moved 4/6 to 5/6: the missing-concept failure was repaired in one round, the ordering failure was not repaired in three, and its violation rate went 0.048 → 0.048 → 0.381, worse on the third attempt than on the first.

Reading the split. The loop closes every failure on the backbone that fills a required slot and one of two on the backbone that does not; the same division appears for the slot itself in the second course. A repair loop is not a separate capability that rescues a weak condition—it inherits the contract's dependence on the backbone. Two cautions. Three successes out of three is a count, not a rate: its exact 95% lower bound is $0.05^{1/3} \approx 0.37$, so the loop is shown to work and not shown to be reliable. And three attempts on one deck produced three different decks with no convergence, so a repair budget cannot rest on the expectation of convergence; it has to be set by cost. The main table therefore keeps 67% as a first-pass figure, and 9/9 is reported as the outcome of a bounded, costed repair on nine decks—not as the pass rate of the method.

S12 Reproducibility

All runs were issued against the vendor's hosted API (`https://api.deepseek.com`) with model id `deepseek-v4-flash`, temperature 0.3, the provider's default thinking mode, and no fixed seed; the model therefore returns different slide text on repeated calls, which is why each cell is resampled three times rather than run once. Slides are emitted as JSON and rendered with `python-pptx`; no image generation model is involved, which is what makes `editability` 1.000 in every condition.

Code, graph, experimental protocol and all raw outputs are available in the `kg-slidegen` repository. The accompanying protocol document records how every number in this paper was produced, and each experimental cell retains three artefacts: the presentation file, the raw model output and a per-item validation report. The teacher ratings are released as the file `paper/human_eval.csv`, one row per deck–rater pair, and `paper/human_eval.py` regenerates the blinded ratings table in the main text from it, together with the ICC values and their bootstrap intervals.